Reflection Report -- SIMS

Student: Marcuel L. Gubaynon

Course: Laravel Web Development

Project: Student Information Management System

Prepared: 12 November 2026

> **Draft based on the project's prompt and verification logs. Review it, add your personal experience where marked, and confirm each statement before submitting.**

1. AI tools and prompt strategy

The project records use of the Copilot SDK in Visual Studio Code, with the model not recorded for most entries, and Codex desktop with GPT-6 for later work. I used short requests to identify a feature, then clarified behavior when the first request left important details open. For example, the dashboard design request was narrowed to existing SIMS routes and real data. The subject schedule request was clarified to include weekdays and a start/end time, with validation for incomplete schedules.

Personal detail to add: Why did I choose these tools? Which one helped me most, and what limitation did I notice?

2. Agent strategy

The repository has role prompts for planning, database work, Laravel implementation, testing, review, and documentation. The project history generally records a primary assistant and no subagents. The role architecture is useful for dividing a large future change and assigning clear handoffs, but I should not claim that a multi-agent workflow was executed when the records do not show one.

Personal detail to add: Which role would have saved me the most effort? Which roles were not necessary for a small change?

3. What worked

Breaking work into features made it possible to refine requirements without replacing the whole app. The project records show tests being added for access boundaries, validation, student data isolation, and CRUD behavior. A useful example of critical review was the student timeline: the system had enrollment and class-schedule data, but no attendance or quiz records, so the implementation used data the app actually stored instead of displaying invented activity.

4. What failed and how it was corrected

A production dashboard request failed because the Wasmer SQLite database did not have an events table. The recorded error was SQLSTATE[HY000]: General error: 1 no such table: events. The correction added a schema check around dashboard event queries and configured migrations in the build flow. The recorded checks showed /login returning 200, /up returning 200, and unauthenticated /dashboard redirecting with 302. The authenticated dashboard was not verified after the last deployment, so that check remains open.

Another iteration involved student login. A forced password-change screen did not match the requested experience; it was removed and a regression assertion was recorded for direct dashboard access. That history is a reminder to confirm the intended user flow before expanding a feature.

5. AI output verification and human judgment

AI suggestions were checked against existing routes, stored data, role boundaries, tests, and real deployment errors. The prompt log records corrections such as using api. route-name prefixes to avoid a web/API naming collision and avoiding reference-page fields that SIMS does not store. I should rerun the current test suite and manually demonstrate the final app before treating historical test results as current evidence.

Student registration and approval update

The latest change adds a student registration form and a pending state. The admin dashboard displays pending accounts for administrators, and student login checks approval before opening the portal. Existing student records can register by matching their stored profile. The approval migration was applied locally and Blade compilation succeeded. I have not yet checked the flow in a browser or run feature tests, so those results must be added after I verify them.

6. Challenges and lessons learned

The main challenge recorded in this project was that local and deployed environments can differ: a local migration can be complete while the deployed SQLite package still lacks a newer table. Exact production error logs helped narrow that problem. Another lesson is that a polished interface must still use real project data and preserve role checks.

Personal detail to add: What was the hardest part for me? What did I learn while explaining or debugging it?

7. What I would improve with two more weeks

I would capture the required screenshots, finish and export the portfolio reports, rerun the full feature suite, manually verify admin/staff/student access, and confirm the authenticated deployed dashboard and Postman API flow. I would also use the documented specialized roles on a future feature and record each real handoff if the tools support it.

Final student review

Before submitting, replace the marked personal-detail prompts with your own experience, verify the tool/model details, add genuine screenshots to the portfolio, and update the verification log with checks you personally rerun. This draft intentionally does not claim unrecorded agent runs, screenshots, AI comparisons, or deployment verification.